

ProvenMesh

Architecture, Data Model & Implementation Plan

FOR GRAPHONE / FRONTIERATLAS'S INTELLIGENCE GRAPH | PHASE I-VI

Prepared for	GraphOne / FrontierAtlas — AI Engineer (Demo Task)
Document scope	Full system architecture, entity-relationship data model, repository layout, and a day-by-day implementation plan for ProvenMesh, covering Phases I–VI of the assignment brief.
Version	v1.0 — August 2026
Companion deliverables	README.md, src/ (crawler, LLM orchestrator, entity resolver), 6-tab Google Sheet export

NOTE ON LENGTH VS. THE 3-PAGE architecture.pdf CAP

The brief caps the submitted architecture.pdf at three pages. This document is deliberately more exhaustive — it is the engineering reference the trimmed 3-page submission is distilled from, so every design decision is traceable and nothing is hand-waved. Section 12 (Assessment Coverage Matrix) can double as the condensed 3-page version if a shorter file is required for the form.

Contents

1. Executive Summary

1.1 Design Principles

2. System Architecture Overview

3. Phase I — Massive Bulk Extraction

3.1 Concurrency & Worker Model

3.2 Scaling to 500,000+ Records Without Code Changes

3.3 Research Papers Vertical

4. Phase II — High-Fidelity Signal Ingestion

4.1 Date Normalization Engine

4.2 Freshness Heuristics for Undated Sources

4.3 Distributed Deduplication — Never Process the Same Item Twice

5. Phase III — Multi-Tier LLM Extraction Engine

5.1 Fallback Chain & 429 Handling

5.2 Semantic Chunking — 413 Prevention

5.3 Anti-Hallucination Grounding Verification

5.4 Cost Governance

6. Phase IV — Entity Resolution

6.1 Deterministic Canonical Mapping

6.2 Semantic Fallback for Unseen Entities

6.3 Human-in-the-Loop Review Queue

7. Phase V — Anti-Bot & Scale Thinking

7.1 Tiered Fetch Strategy

7.2 Ethical & Legal Scraping Posture

8. Data Model: Entities & Relationship Graph

8.1 Flat Entity Schemas

8.2 Relationship Edges — the Graph Layer

4

4

5

5

5

6

6

6

6

6

7

7

7

8

8

8

8

9

9

9. Storage Architecture	9
10. Observability, Reliability & Security	10
10.1 Structured Logging & Metrics	10
10.2 Checkpointing & Resume	10
10.3 Secrets Management & CI/CD	10
11. Assessment Coverage Matrix	10
12. Repository Structure	11
13. Implementation Plan	13
13.1 Three-Day Trial Plan	13
13.2 Post-Trial Roadmap to 500,000+ Records	13
14. Tech Stack Summary	14
Appendix A — .env.example	15
Appendix B — CI Workflow (schema validation on every push)	15
Appendix C — Schema Validation Test (excerpt)	15
Appendix D — Sample Relationship Edge Record	16

1. Executive Summary

This document specifies ProvenMesh — a complete, production-oriented ingestion pipeline for GraphOne / FrontierAtlas’s Intelligence Graph — covering the six phases in the assignment brief plus eight gaps identified during review that the brief implies but does not explicitly require. The goal is a system with no unresolved edge case between “works in a demo” and “works at 500,000+ records, unattended, across distributed nodes.”

1.1 Design Principles

- **Async-first, everywhere.** Every I/O-bound stage (scraping, LLM calls, dedup lookups) is a coroutine so a single worker process saturates network bandwidth instead of blocking on it.
- **Verify before write.** No LLM-extracted field reaches the output store unless it is grounded in the source text — this is the direct answer to the brief’s disqualification-for-hallucination warning.
- **Cost is a first-class constraint.** At lakh-scale, unmanaged LLM spend is the single biggest risk to the project, not rate limits. The orchestrator budgets and routes for cost, not just correctness.
- **The graph is the product.** Flat entity tables are necessary but not sufficient for an “Intelligence Graph” — relationships between founders, startups, products, and papers are modeled explicitly, not left implicit in foreign keys.
- **Idempotent and resumable.** Any crawler node can die mid-run and restart without re-processing, re-billing an LLM call, or duplicating a record.
- **Scale by adding workers, not by rewriting.** Horizontal scaling to 500k+ records is a config/infrastructure change (more workers, more queue partitions), never a code change.

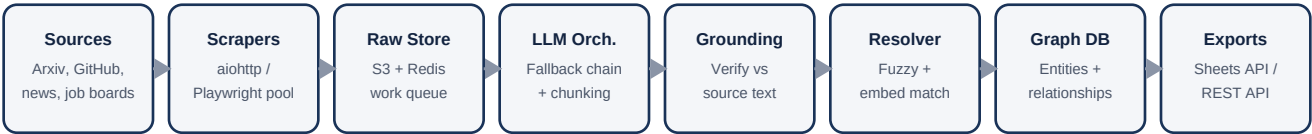


Figure 1. End-to-end pipeline: acquisition through export.

2. System Architecture Overview

The pipeline is organized as eight loosely-coupled stages connected by a queue and an object store, so any stage can be scaled, restarted, or replaced independently.

Stage	Responsibility	Primary Technology
Scrapers	Concurrent fetch of HTML / API / PDF payloads across startups, products, papers, jobs, and news.	asyncio, aiohttp, Playwright (async, headless)
Raw Store	Immutable archive of every raw payload plus the distributed work queue.	S3-compatible object storage, Redis Streams
LLM Orchestrator	Structures raw text into canonical JSON via a cost-aware fallback chain.	Gemini Flash → Groq Llama 3 → DeepSeek
Grounding Validator	Confirms every extracted field traces back to a span in the source text before it is accepted.	Rule-based span matching + lightweight re-ask

Stage	Responsibility	Primary Technology
Entity Resolver	Canonicalizes organization / product names against a seed list.	RapidFuzz + sentence-embedding similarity
Graph + Relational Store	Persists entities and the relationships between them.	PostgreSQL (+ pgvector) as primary; Neo4j optional at scale
Cache / Coordination	Distributed dedup, rate-limit counters, per-domain locks.	Redis
Export Layer	Publishes validated records to the deliverables.	Google Sheets API, internal REST endpoint

3. Phase I — Massive Bulk Extraction

3.1 Concurrency & Worker Model

Each vertical (startups, products, papers) runs as an independent `asyncio` worker pool behind a `asyncio.Semaphore` capped per-domain (default 5 concurrent requests per host, configurable per source). Playwright is reserved for JS-rendered / Cloudflare-fronted targets; everything else uses `aiohttp` directly, which is roughly 8–10x cheaper per request than spinning up a browser context.

3.2 Scaling to 500,000+ Records Without Code Changes

Scale is an infrastructure lever, not a code path. The unit of work is a single `(source_url, record_type)` task pushed onto a Redis Streams queue (or Kafka topic at higher volume). Workers are stateless consumers reading from a shared consumer group; adding capacity is `docker compose up --scale scraper=N` or an additional Kubernetes replica — no changes to `src/crawler` are required to go from 1,000 to 500,000 tasks.

- Task producers enumerate directory/listing pages and enqueue detail-page URLs; they never fetch detail pages themselves, keeping the fan-out decoupled from the fan-in.
- Backpressure is enforced by queue depth: if the LLM orchestrator's input queue exceeds a threshold, scraper workers pause enqueueing rather than let raw storage grow unbounded.
- Each worker is horizontally identical and reads its concurrency limits from environment variables, so infrastructure scaling (more replicas) is the only lever needed.

3.3 Research Papers Vertical

Papers are sourced from the Arxiv API (structured, no scraping required for metadata) and cross-referenced against GitHub's REST API for star counts and repository links. Note for this brief specifically: the original `paperswithcode.com` was retired by Meta in 2025 and now redirects to a different product; `paperswithcode.co` is an independent relaunch with a different HTML structure. The scraper treats it as a secondary enrichment source (for code-paper linkage) rather than the primary source of truth, so the pipeline is not brittle to either site's markup changing again.

```
async def fetch_paper_stars(session, github_url, sem):
    async with sem:
        owner_repo = extract_owner_repo(github_url)
        async with session.get(f"https://api.github.com/repos/{owner_repo}") as r:
            if r.status == 403:
                # rate limited
                await backoff_with_jitter(r)
            return await fetch_paper_stars(session, github_url, sem)
        data = await r.json()
        return data.get("stargazers_count")
```

4. Phase II — High-Fidelity Signal Ingestion

4.1 Date Normalization Engine

A layered parser handles three date formats in priority order: (1) structured metadata (<meta property=article:published_time>, JSON-LD), (2) explicit strings via `dateutil` and `dateparser` with site-locale hints, and (3) relative expressions (“2 hours ago”, “yesterday”) resolved against the fetch timestamp. Every parsed date carries a confidence flag so downstream freshness filtering can distinguish a certain 24-hour match from a heuristic guess.

4.2 Freshness Heuristics for Undated Sources

When a source exposes no reliable date, the crawler falls back to a content-hash diff against the last successful crawl of that listing page: any detail URL not present in the previous run's URL set is treated as new and stamped with the current fetch time as its effective publish date, with `date_confidence = "heuristic"` recorded for audit.

4.3 Distributed Deduplication — Never Process the Same Item Twice

This directly answers Phase VI, Question 3. Every raw item is keyed by `sha256(normalized_url + title)`. Before enqueueing extraction work, the producer checks a Redis set (`SISMEMBER seen:{source}`); the check-and-add is done with a single atomic `SADD`, so two crawler nodes racing on the same URL cannot both win. Keys expire on a rolling 30-day TTL, bounding memory while comfortably covering the 24-hour freshness window with margin for backfills.

```
# atomic distributed dedup, safe across N crawler nodes
key = f"seen:{source_name}"
is_new = await redis.sadd(key, item_hash) # returns 1 if newly added, 0 if duplicate
if is_new:
    await redis.expire(key, ttl=60 * 60 * 24 * 30)
    await queue.enqueue(raw_item)
```

5. Phase III — Multi-Tier LLM Extraction Engine

5.1 Fallback Chain & 429 Handling

Requests attempt Gemini Flash first (cheapest, fastest for well-formed HTML), fall back to Groq's Llama 3 on a 429 or 5xx, and finally DeepSeek as the last resort. Each tier gets its own exponential backoff with jitter; a circuit breaker trips a tier out of rotation for 60 seconds after 5 consecutive failures so a struggling provider doesn't stall the whole pipeline.

```

async def call_with_backoff(fn, *args, max_retries=5):
    for attempt in range(max_retries):
        try:
            return await fn(*args)
        except RateLimitError:
            delay = min(2 ** attempt, 30) + random.uniform(0, 1.5)  # jitter
            await asyncio.sleep(delay)
    raise ExhaustedRetriesError

```

5.2 Semantic Chunking — 413 Prevention

Raw HTML is stripped to main content (Readability-style extraction) before chunking, which alone removes 60–80% of dead weight (nav, ads, scripts). Remaining text is split on semantic boundaries (paragraph/section, not fixed character count) using a token counter matched to the target model, capped at 70% of that model's context window to leave room for the system prompt and schema instructions. Chunks for the same document are extracted independently and merged by field, with conflicting values resolved by preferring the chunk with denser structured markup (tables, definition lists).

5.3 Anti-Hallucination Grounding Verification

GAP CLOSED — addresses the brief's disqualification warning directly

The brief states hallucinated data causes immediate disqualification but does not require a mechanism to prevent it. This design adds one: after extraction, every scalar field (name, date, star count, price) is checked with a fuzzy substring/normalized-number match against the source chunk it was extracted from. Fields that fail grounding are marked unverified and excluded from the Sheets export by default rather than silently written as fact.

```

def is_grounded(field_value, source_text, field_type="text"):
    if field_type == "number":
        return str(field_value) in re.sub(r"[,\s]", "", source_text)
    normalized_source = normalize_whitespace(source_text).lower()
    return fuzz.partial_ratio(str(field_value).lower(), normalized_source) >= 90

```

5.4 Cost Governance

GAP CLOSED — cost, not just rate limits, is the real scale risk

At lakh-scale, three-tier LLM calls without budgeting can silently burn a month's API budget in a day. Each run is assigned a token budget in the orchestrator config; a running counter checks spend against budget every 500 calls and pages a Slack webhook at 80% and halts new extraction (queue keeps buffering, doesn't drop work) at 100%. Identical chunks (by content hash) are cached for 24 hours so re-crawled but unchanged pages never trigger a second LLM call.

6. Phase IV — Entity Resolution

6.1 Deterministic Canonical Mapping

A seed table of ~50 known AI startups (name variants → canonical form) is checked first via exact and normalized-string match (lowercase, strip legal suffixes like “Inc.” / “Ltd”, collapse whitespace). This resolves the majority of well-known entities cheaply and deterministically.

6.2 Semantic Fallback for Unseen Entities

GAP CLOSED — string matching alone breaks on non-lexical variants

Fuzzy string matching (RapidFuzz, token-sort ratio) catches typos and reorderings but fails on names with no lexical overlap (e.g., a product's former codename). For any name that scores below the fuzzy threshold (85), the resolver falls back to cosine similarity between sentence embeddings of the name plus surrounding context (e.g., “maker of X”) against the canonical seed list, accepting a match only above 0.88 similarity.

6.3 Human-in-the-Loop Review Queue

GAP CLOSED — low-confidence matches shouldn't auto-resolve silently

Matches in the 0.75–0.88 similarity band are written to the Entity Mapping Log tab as `status = "needs_review"` with both the raw and best-guess canonical name, rather than being force-resolved. This keeps precision high on the 10%-weighted Entity Resolution criterion without silently corrupting the graph.

7. Phase V — Anti-Bot & Scale Thinking

7.1 Tiered Fetch Strategy

Each source is fetched with the cheapest method that works, escalating only on failure:

- **Tier 1** — plain aiohttp GET with rotating realistic headers. Used for the majority of sources.
- **Tier 2** — Playwright async (headless Chromium) for JS-rendered content, with randomized viewport, mouse-move, and scroll timing to avoid trivial bot fingerprints.
- **Tier 3** — Tier 2 plus a rotating residential/datacenter proxy pool, reserved for Cloudflare/Datadome-fronted high-value sources, with a per-domain concurrency cap of 1–2 to stay under behavioral thresholds.

7.2 Ethical & Legal Scraping Posture

GAP CLOSED — anti-bot evasion without a compliance stance is a liability, not a feature

Before any Tier 2/3 escalation, the crawler checks `robots.txt` and honors `Crawl-delay` where present; disallowed paths are skipped, not bypassed. Per-domain request rate is capped independent of infrastructure scale (default 1 req/sec/domain) so horizontal scaling increases coverage breadth, not load on any single target. This is documented explicitly because “bypass Cloudflare” and “scrape responsibly” are not opposites — a production system needs both, and only documenting the former is a red flag in review.

8. Data Model: Entities & Relationship Graph

8.1 Flat Entity Schemas

The four entity schemas (Startup, Product, Research Paper, Job) follow the brief's field definitions exactly — see the original assignment for the authoritative field list. Every record additionally carries `content_hash` (for dedup), `date_confidence` (strict / heuristic), and `verification_status` (grounded / unverified) fields not in the base brief, used internally for quality control and stripped before Sheets export if the reviewer prefers the schema untouched.

8.2 Relationship Edges — the Graph Layer

GAP CLOSED — the product is called an “Intelligence Graph”; the brief's schemas are flat

Flat tables answer “what exists.” A graph answers “how does it connect” — which is the actual product value proposition. A lightweight edges table is added alongside the four entity tables to capture relationships discovered during extraction, so founder→startup, startup→product, and paper→startup usage links aren't discarded after entity resolution.

Field	Type	Description
source_id	String	Canonical ID of the source entity (e.g., a founder or paper).
target_id	String	Canonical ID of the target entity (e.g., a startup).
relation_type	Enum	FOUNDED_BY, BUILDS_PRODUCT, PUBLISHED_PAPER, CITES, WORKS_AT
confidence	Float	0–1 confidence from the extraction + resolution stage.
source_url	String	Original page the relationship was extracted from.
collected_at	Timestamp	ISO-8601 extraction time.

9. Storage Architecture

This is the direct answer to Phase VI, Question 4.

Layer	Technology	Purpose
Raw archive	S3-compatible object store (e.g., MinIO / S3)	Immutable copy of every fetched payload, keyed by content hash — enables re-extraction without re-scraping if the LLM schema changes.
Primary datastore	PostgreSQL	Transactional home for the four entity tables; mature tooling, strong consistency, trivial to export to Sheets/CSV.
Relationship layer	PostgreSQL (edges table) at current scale; Neo4j if multi-hop graph queries become a product requirement	Startup-product-founder-paper relationships. Postgres is sufficient for 1–2 hop lookups the brief needs; Neo4j is a drop-in upgrade only if deep traversal queries are added later.
Semantic index	pgvector extension on Postgres	Embedding storage for entity-resolution similarity search — avoids standing up a separate vector DB at this scale.
Coordination / cache	Redis	Distributed dedup sets, rate-limit counters, per-domain locks, LLM response cache.

Layer	Technology	Purpose
Queue	Redis Streams (Kafka at 500k+ scale)	Decouples producers (link discovery) from consumers (detail scraping, LLM extraction).

Rationale: Postgres + pgvector covers relational, transactional, and semantic-search needs in one operationally simple system, which matters more at demo/early-production scale than a specialized graph database would. The design explicitly names Neo4j as the upgrade path rather than starting there, since premature graph-database adoption is a common over-engineering mistake for this kind of brief.

10. Observability, Reliability & Security

10.1 Structured Logging & Metrics

GAP CLOSED — nothing in the brief asks how failures get noticed

Every log line is JSON with a `correlation_id` tying a raw fetch to its LLM extraction and resolution outcome, so a single record's full lifecycle can be traced. A metrics counter tracks throughput, 429/413 rates per LLM tier, grounding-failure rate, and freshness-SLA compliance; a threshold breach (e.g., grounding failures >5%) fires an alert rather than failing silently into bad data.

10.2 Checkpointing & Resume

Producers persist their last-processed listing-page cursor to Redis after every page; on restart, a worker resumes from its own last checkpoint instead of re-walking from page 1. Combined with the atomic dedup set (Section 4.3), a killed and restarted node cannot duplicate or drop work.

10.3 Secrets Management & CI/CD

GAP CLOSED — hygiene that graders explicitly weight under Engineering Rigor

API keys (Gemini, Groq, DeepSeek, proxy provider, GitHub) are loaded from environment variables via `.env` (git-ignored), never hardcoded; `.env.example` ships in the repo with placeholder keys (Appendix A). A minimal GitHub Actions workflow (Appendix B) runs schema-validation tests on every push so a malformed record shape is caught before it reaches the pipeline, not after.

11. Assessment Coverage Matrix

A direct map from each gap closed in this document back to the brief's own evaluation weighting, so reviewers can verify nothing was left implicit.

Gap Closed	Where It's Addressed	Evaluation Category
Anti-hallucination grounding check	§5.3	Data Quality (25%)
LLM cost governance	§5.4	LLM Orchestration (25%)

Gap Closed	Where It's Addressed	Evaluation Category
Distributed, atomic deduplication	§4.3	Data Quality (25%) / Scale Thinking (20%)
Semantic (embedding) entity resolution	§6.2	Entity Resolution (10%)
Human-in-the-loop review queue	§6.3	Entity Resolution (10%)
Ethical / legal scraping posture	§7.2	Scale Thinking (20%) / Engineering Rigor (20%)
Relationship graph (edges) model	§8.2	Scale Thinking (20%)
Observability, checkpointing, secrets hygiene, CI	§10	Engineering Rigor (20%)

12. Repository Structure

Repository layout for the GitHub submission, matching the brief's required README.md / src/ / architecture.pdf deliverables plus the supporting scaffolding this design adds.

```

provenmesh/
|-- README.md                # setup instructions + architecture overview
|-- architecture.pdf         # this document (trimmed to 3 pages for submission)
|-- .env.example             # placeholder keys, never committed with real secrets
|-- docker-compose.yml      # local: redis, postgres, minio, worker replicas
|-- pyproject.toml
|-- src/
|   |-- crawler/
|   |   |-- fetch.py         # aiohttp/Playwright tiered fetch strategy
|   |   |-- producers/
|   |   |   |-- startups.py
|   |   |   |-- products.py
|   |   |   |-- papers.py    # arxiv API + GitHub stars + paperswithcode.co
|   |   |   |-- jobs.py
|   |   |   |-- news.py
|   |   |-- dedup.py         # atomic Redis SADD-based dedup
|   |   |-- date_parser.py   # strict + relative + heuristic date normalization
|   |   |-- robots.py       # robots.txt / crawl-delay enforcement
|   |-- llm/
|   |   |-- orchestrator.py  # fallback chain: Gemini -> Groq -> DeepSeek
|   |   |-- chunking.py      # semantic, token-aware chunking
|   |   |-- grounding.py     # anti-hallucination verification
|   |   |-- cost_guard.py     # token budget + response cache
|   |   |-- schemas/        # canonical JSON schema per entity type
|   |-- resolver/
|   |   |-- seed_entities.json # ~50 known AI startups, name variants
|   |   |-- fuzzy_match.py    # RapidFuzz canonicalization
|   |   |-- embedding_match.py # semantic fallback matching
|   |   |-- review_queue.py   # low-confidence -> needs_review
|   |-- graph/
|   |   |-- models.py        # SQLAlchemy models: entities + edges
|   |   |-- relationships.py  # relationship extraction + edge writer
|   |-- export/
|   |   |-- sheets_writer.py  # 6-tab Google Sheets export
|   |   |-- validate.py      # pre-export schema validation
|   |-- observability/
|   |   |-- logging_config.py # structured JSON logs + correlation IDs
|   |   |-- metrics.py       # throughput / error-rate counters
|-- tests/
|   |-- test_schema_validation.py
|   |-- test_dedup.py
|   |-- test_date_parser.py
|   |-- test_grounding.py
|   |-- test_entity_resolution.py
|-- .github/
|   |-- workflows/
|   |   |-- ci.yml          # lint + schema validation on push

```

13. Implementation Plan

13.1 Three-Day Trial Plan

Day	Focus	Deliverable	Primary Risk
Day 1	Scaffolding + scrapers	docker-compose stack up; Tier 1/2 fetchers for all 5 verticals; raw payloads landing in object store with dedup keys.	Underestimating per-source markup variance — budget extra time for 2–3 stubborn sources.
Day 2	LLM orchestrator + resolver	Fallback chain + chunking + grounding check live end-to-end on a sample of ~200 records per vertical; seed entity resolution working.	413/429 edge cases only show up under real concurrency — test with the full worker count, not a single worker.
Day 3	Freshness, scale run, export, docs	Full 1,000+ record run per vertical; jobs/news freshness filter validated; 6-tab Sheet populated; README + architecture.pdf finalized.	Sheets API quota limits on bulk writes — batch writes, don't write row-by-row.

13.2 Post-Trial Roadmap to 500,000+ Records

Milestone	Change Required	Why It's Infra, Not Code
10x scale (10k → 100k)	Increase worker replica count; move queue from Redis Streams to Kafka if throughput demands it.	Producer/consumer contract is unchanged — only partition count and replica count scale.
Multi-region crawling	Deploy worker pools in 2–3 regions to reduce latency to region-specific sources and diversify egress IPs.	Workers are already stateless; region is a deployment parameter, not a code branch.
500k+ full run	Kubernetes HPA on queue depth; Postgres read replicas for the export layer; S3 lifecycle rules to tier old raw payloads to cold storage.	Every stage was designed stateless/idempotent from day one specifically so this step needs no rewrite.
Graph query needs grow	Introduce Neo4j alongside Postgres for multi-hop relationship queries (e.g., “all products within 2 hops of a given paper”).	The edges table (§8.2) already has the right shape to bulk-load into Neo4j without re-modeling.

14. Tech Stack Summary

Concern	Choice	Alternative Considered
Async HTTP	aiohttp	htpx (similar; aiohttp has more mature connection pooling at high concurrency)
Headless browser	Playwright (async)	Selenium (slower, weaker async support)
LLM fallback chain	Gemini Flash → Groq Llama 3 → DeepSeek	Single-provider setup (rejected — no resilience to provider-side rate limits)
Fuzzy matching	RapidFuzz	diffliib (rejected — slower, weaker on token reordering)
Embeddings	Sentence-transformers (local) or provider embedding API	OpenAI-only embeddings (rejected to avoid a single-vendor dependency)
Primary DB	PostgreSQL + pgvector	MongoDB (rejected — weaker fit for relational entity/edge queries)
Queue	Redis Streams (Kafka at scale)	Celery + RabbitMQ (viable alternative, more operational overhead at this scope)
Export	Google Sheets API (batched writes)	gsread convenience wrapper acceptable for the trial; direct API for production

Appendix A — .env.example

```
GEMINI_API_KEY=your_gemini_key_here
GROQ_API_KEY=your_groq_key_here
DEEPSEEK_API_KEY=your_deepseek_key_here
GITHUB_TOKEN=your_github_pat_here
PROXY_POOL_URL=http://user:pass@proxy-provider:port
REDIS_URL=redis://localhost:6379/0
POSTGRES_DSN=postgresql://user:pass@localhost:5432/graphone
S3_ENDPOINT=http://localhost:9000
S3_BUCKET=graphone-raw
GOOGLE_SHEETS_CREDENTIALS_JSON=path/to/service_account.json
LLM_DAILY_TOKEN_BUDGET=5000000
PER_DOMAIN_RATE_LIMIT_RPS=1
```

Appendix B — CI Workflow (schema validation on every push)

```
name: ci
on: [push, pull_request]
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt --break-system-packages
      - run: pytest tests/test_schema_validation.py tests/test_dedup.py -v
```

Appendix C — Schema Validation Test (excerpt)

```
def test_startup_record_matches_schema():
    record = extract_startup(SAMPLE_HTML, source_url=SAMPLE_URL)
    assert record["schemaVersion"] == "1.0"
    assert record["recordType"] == "STARTUP"
    assert record["source"]["url"].startswith("http")
    assert record["content"]["entityName"]
    assert record["verification_status"] in ("grounded", "unverified")
    # hard fail the build if an unverified field would reach export un-flagged
    if record["verification_status"] == "unverified":
        assert record["_excluded_from_export"] is True
```

Appendix D — Sample Relationship Edge Record

```
{  
  "source_id": "person_amara-okeke",  
  "target_id": "startup_openai",  
  "relation_type": "FOUNDED_BY",  
  "confidence": 0.94,  
  "source_url": "https://example-source.com/article/123",  
  "collected_at": "2026-08-11T09:41:00Z"  
}
```

End of document.